ESTRUTURA DE DADOS

PYTHON

Profª Christine Vieira

Instalação do Python e VSCode

https://youtu.be/7Kpd6eprz4k

```python
idade = int(input("Digite sua idade: "))
sexo = input("Digite seu sexo (M para masculino, F para feminino): ")

# Verificando se a pessoa deve se alistar
if idade == 18 and sexo.upper() == "M":
    print("Caro", nome + ",", "você deve se alistar!")
elif idade < 18:
    print("Caro", nome + ",", "você ainda não tem idade para se alistar.")
else:
    print("Caro", nome + ",", "você não precisa se alistar neste momento.")
```

Faça um programa que lei um nome de usuário e a sua senha e não aceite a senha igual ao nome do usuário, mostrando uma mensagem de erro e voltando a pedir as informações.

```
while True:
    # Solicitando nome de usuário e senha
    usuario = input("Digite seu nome de usuário: ")
    senha = input("Digite sua senha: ")

    # Verificando se a senha é igual ao nome de usuário
    if senha == usuario:
        print("Erro: A senha não pode ser igual ao nome de usuário. Por favor, tente
    else:
        print("Cadastro realizado com sucesso!\n")
        break
```

Menu e funções

```python
def opcao1():
    print("Você escolheu a opção 1.")

def opcao2():
    print("Você escolheu a opção 2.")

def main():
    while True:
        print("\nMenu:")
        print("1- Opção 1")
        print("2- Opção 2")
        print("0- Sair")

        escolha = input("Escolha uma opção: ")
```

```python
        if escolha == "1":
            opcao1()
        elif escolha == "2":
            opcao2()
        elif escolha == "0":
            print("Saindo do programa...")
            break
        else:
            print("Opção inválida. Por favor, escolha uma opção válida.")

if __name__ == "__main__":
    main()
```

Menu e funções

```python
def opcao1():
    print("Você escolheu a opção 1.")

def opcao2():
    print("Você escolheu a opção 2.")

def main():
    while True:
        print("\nMenu:")
        print("1- Opção 1")
        print("2- Opção 2")
        print("0- Sair")

        escolha = input("Escolha uma opção: ")
```

```python
        if escolha == "1
            opcao1()
        elif escolha ==
            opcao2()
        elif escolha ==
            print("Saind
            break
        else:
            print("Opção
                                                    ")

if __name__ == "__main__":
    main()
```

Ao escrever if __name__ == "__main__": main(), você está garantindo que a função main() só será chamada se este arquivo for executado diretamente como o programa principal. Isso é útil para separar a lógica principal do programa em uma função main(), que só será executada quando o arquivo for executado diretamente, mas não quando for importado como um módulo em outro script. Isso ajuda a modularizar e organizar melhor o código Python.

Vetor, Array e Lista

Em Python, o termo "vetor" é frequentemente usado de forma com o termo "lista".

Em um sentido mais técnico, um vetor é uma coleção ordenada de elementos do mesmo tipo de dados.

No entanto, em Python, as listas são tão flexíveis que geralmente são usadas para representar vetores, matrizes e outras estruturas de dados multidimensionais.

Uma lista em Python pode conter elementos de tipos de dados diferentes e pode crescer ou diminuir de tamanho dinamicamente, o que se assemelha ao comportamento de um vetor em outras linguagens de programação.

Por exemplo, uma lista em Python pode ser usada como um vetor da seguinte maneira:

vetor = [1, 2, 3, 4, 5]

```
vetor = [1, 2, 3, 4, 5]
```

```
numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Ao falar sobre vetores em Python, geralmente estamos nos referindo a listas.

E as operações que realizamos com vetores em outras linguagens, como acesso a elementos, adição de elementos, remoção de elementos, etc., são todas realizadas utilizando as operações disponíveis para listas em Python.

Em Python, uma lista é uma estrutura de dados que permite armazenar uma coleção ordenada de elementos. É uma das estruturas de dados mais versáteis e amplamente utilizadas na linguagem. As listas em Python são mutáveis, o que significa que você pode adicionar, remover e modificar elementos depois que a lista é criada.

As listas são definidas utilizando colchetes [], e os elementos são separados por vírgulas. Os elementos de uma lista podem ser de diferentes tipos de dados, incluindo números, strings, outras listas, ou até mesmo objetos mais complexos.

```
lista = [1, 2, 3, 4, 5]
```

Algumas operações comuns que podem ser realizadas com listas incluem:

Acesso aos elementos individuais utilizando índices: lista[0] retorna o primeiro elemento da lista.
Adição de elementos ao final da lista: **lista.append(6)** adicionaria o número 6 ao final da lista.
Remoção de elementos: **lista.remove(3)** removeria o número 3 da lista.
Iteração sobre os elementos da lista utilizando loops for.

E muitas outras operações úteis, como ordenação, fatiamento, e concatenação de listas.

As listas são uma parte fundamental da linguagem Python e são amplamente utilizadas em muitos contextos diferentes, desde manipulação de dados simples até implementação de algoritmos mais complexos.

```python
# Criando uma lista com tipos diferentes
lista = ["Hello", 42, 3.14, True, [1, 2, 3], {"nome": "João", "idade": 30}]

# Imprimindo a lista
print(lista)
```

Neste exemplo, a lista **lista** contém strings ("Hello"), inteiros (42), floats (3.14), booleanos (True), outra lista ([1, 2, 3]), e um **dicionário** ({"nome": "João", "idade": 30}).

Todos esses tipos de dados diferentes são armazenados em uma única lista. Isso demonstra a flexibilidade das listas em Python.

Dicionário

```python
# Criando uma lista com dicionários
lista_pessoas = [
    {"nome": "João", "idade": 30, "cidade": "São Paulo"},
    {"nome": "Maria", "idade": 25, "cidade": "Rio de Janeiro"},
    {"nome": "Pedro", "idade": 35, "cidade": "Belo Horizonte"}
]

# Iterando sobre a lista e acessando os valores dos dicionários
for pessoa in lista_pessoas:
    print(f"Nome: {pessoa['nome']}, Idade: {pessoa['idade']}, Cidade: {pessoa['cidade
```

```python
# Criando uma lista vazia para armazenar os dicionários
lista_pessoas = []

# Pedindo ao usuário para informar os dados das pessoas
while True:
    nome = input("Informe o nome da pessoa (ou digite 'O' para encerrar): ")
    if nome == 'O':
        break

    idade = int(input("Informe a idade da pessoa: "))
    cidade = input("Informe a cidade da pessoa: ")

    # Criando um dicionário com os dados informados e adicionando-o à lista
    pessoa = {"nome": nome, "idade": idade, "cidade": cidade}
    lista_pessoas.append(pessoa)
```

```
# Imprimindo a lista de pessoas
print("\nLista de pessoas:")
for pessoa in lista_pessoas:
    print(f"Nome: {pessoa['nome']}, Idade: {pessoa['idade']}, Cidade: {pessoa['cidade']}")
```

Limpar a Tela

Para consoles baseados no Windows:

Você pode usar o comando de sistema `cls`. Aqui está um exemplo de como fazer isso:

python Copy code

```python
import os

# Limpar a tela
os.system('cls')
```

Criar um novo programa.

1- Inserir elementos em uma lista
2- Pesquisar um elemento na lista
3- Excluir um elemento da lista
4 – Imprimir a lista

Inserir elementos em uma Lista – append – Parte 1

```python
# Função para inserir elementos em uma lista
def inserir_elemento(lista, elemento):
    lista.append(elemento)
    print(f"Elemento {elemento} inserido na lista.")

# Função principal
def main():
    lista = []

    while True:
        print("\n1- Inserir elemento")
        print("0- Encerrar o programa")

        opcao = input("Escolha uma opção: ")
```

Inserir elementos em uma Lista – append – Parte 2

```
        if opcao == "1":
            elemento = input("Digite o elemento a ser inserido na lista: ")
            inserir_elemento(lista, elemento)
        elif opcao == "0":
            print("Encerrando o programa...")
            break
        else:
            print("Opção inválida. Por favor, escolha uma opção válida.")

    print("Lista final:", lista)

if __name__ == "__main__":
    main()
```

Inserir elementos em uma Lista usando a posição – insert - Parte 1

```
def inserir_elemento_posicao(lista, elemento, posicao):
    if 0 <= posicao <= len(lista):
        lista.insert(posicao, elemento)
        print(f"Elemento '{elemento}' inserido na posição {posicao}.")
    else:
        print("Posição inválida.")

def main():
    lista = []

    while True:
        print("Lista atual:", lista)
        opcao = input("\nDigite o elemento a ser inserido (ou 'X' para encerrar): ")
```

Inserir elementos em uma Lista – insert - Parte 2

```
        if opcao.upper() == "X":
            print("Encerrando o programa...")
            break

        posicao = input("Digite a posição onde o elemento deve ser inserido: ")

        try:
            posicao = int(posicao)
            inserir_elemento_posicao(lista, opcao, posicao)
        except ValueError:
            print("Posição inválida. Por favor, insira um número ou 'X' para encerrar

if __name__ == "__main__":
    main()
```

```python
lista = [1, 2, 3]
lista.insert(1, 10)  # Insere o valor 10 na posição 1
print(lista)  # Saída: [1, 10, 2, 3]

lista.append(20)  # Adiciona o valor 20 ao final da lista
print(lista)  # Saída: [1, 10, 2, 3, 20]
```

Subscrever um elemento na Lista

```
lista = [1, 2, 3, 4, 5]
lista[2] = 10
print(lista)  # Saída: [1, 2, 10, 4, 5]
```

Pesquisar elementos em uma Lista – Parte 1

```python
# Função para pesquisar um elemento na lista
def pesquisar_elemento(lista, elemento):
    if elemento in lista:
        print(f"O elemento '{elemento}' está na lista.")
    else:
        print(f"O elemento '{elemento}' não está na lista.")

# Função principal
def main():
    lista = [10, 20, 30, 40, 50]

    print("Bem-vindo! Este programa permite pesquisar um elemento em uma lista.")

    while True:
        elemento = input("\nDigite o elemento a ser pesquisado na lista (ou 'O' para
```

Pesquisar elementos em uma Lista – Parte 2

```
        if elemento == "0":
            print("Encerrando o programa...")
            break

        pesquisar_elemento(lista, elemento)

if __name__ == "__main__":
    main()
```

Pesquisar elementos em uma Lista por uma posição – Parte 1

```python
def pesquisar_por_posicao(lista, posicao):
    if 0 <= posicao < len(lista):
        print(f"O elemento na posição {posicao} é '{lista[posicao]}'.")
    else:
        print("Posição inválida.")

def main():
    lista = [10, 20, 30, 40, 50]

    print("Lista atual:", lista)
```

Pesquisar elementos em uma Lista por uma posição – Parte 2

```
    while True:
        posicao = input("Digite a posição do elemento a ser pesquisado (ou 'X' para e

        if posicao.upper() == "X":
            print("Encerrando o programa...")
            break

        try:
            posicao = int(posicao)
            pesquisar_por_posicao(lista, posicao)
        except ValueError:
            print("Entrada inválida. Por favor, insira um número ou 'X' para encerrar

if __name__ == "__main__":
    main()
```

Excluir elementos em uma Lista – remove – Parte 1

```
# Função para excluir um elemento da lista
def excluir_elemento(lista, elemento):
    if elemento in lista:
        lista.remove(elemento)
        print(f"Elemento '{elemento}' removido da lista.")
    else:
        print(f"O elemento '{elemento}' não está na lista.")

# Função principal
def main():
    lista = [10, 20, 30, 40, 50]

    print("Bem-vindo! Este programa permite excluir um elemento de uma lista.")

    while True:
        elemento = input("\nDigite elemento a ser removido da lista (ou 'O' para e
```

Excluir elementos em uma Lista – remove – Parte 2

```
        if elemento == "0":
            print("Encerrando o programa...")
            break

        excluir_elemento(lista, elemento)

    print("\nLista final:", lista)

if __name__ == "__main__":
    main()
```

Excluir elementos em uma Lista pela posição – pop – Parte 1

```
def excluir_por_posicao(lista, posicao):
    if 0 <= posicao < len(lista):
        elemento_removido = lista.pop(posicao)
        print(f"Elemento '{elemento_removido}' na posição {posicao} foi removido da
    else:
        print("Posição inválida.")

def main():
    lista = [10, 20, 30, 40, 50]

    print("Lista atual:", lista)
```

Excluir elementos em uma Lista pela posição – pop – Parte 2

```
    while True:
        posicao = input("Digite a posição do elemento a ser removido (ou 'X' para e

        if posicao.upper() == "X":
            print("Encerrando o programa...")
            break

        try:
            posicao = int(posicao)
            excluir_por_posicao(lista, posicao)
            print("Lista atual:", lista)
        except ValueError:
            print("Entrada inválida. Por favor, insira um número ou 'X' para encerr

if __name__ == "__main__":
    main()
```

Imprimir uma Lista

```python
# Função para imprimir uma lista
def imprimir_lista(lista):
    print("Lista:", lista)

# Função principal
def main():
    lista = [10, 20, 30, 40, 50]

    print("Bem-vindo! Este programa permite imprimir uma lista.")

    imprimir_lista(lista)

if __name__ == "__main__":
    main()
```

Matriz

Em Python, você pode criar uma matriz usando listas de listas. Cada lista interna representa uma linha da matriz.

```python
# Definindo uma matriz como uma lista de listas
matriz = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

# Imprimindo a matriz
for linha in matriz:
    print(linha)
```

Solicitar ao usuário para inserir um elemento na Matriz – Parte 1

```python
# Função para criar uma matriz com base nas entradas do usuário
def criar_matriz(linhas, colunas):
    matriz = []
    for i in range(linhas):
        linha = []
        for j in range(colunas):
            elemento = int(input(f"Informe o elemento da posição ({i+1}, {j+1}): "))
            linha.append(elemento)
        matriz.append(linha)
    return matriz

# Função para imprimir a matriz
def imprimir_matriz(matriz):
    for linha in matriz:
        print(linha)
```

Solicitar ao usuário para inserir um elemento na Matriz – Parte 2

```python
# Função principal
def main():
    linhas = int(input("Informe o número de linhas da matriz: "))
    colunas = int(input("Informe o número de colunas da matriz: "))

    print("Informe os elementos da matriz:")
    matriz = criar_matriz(linhas, colunas)

    print("\nMatriz inserida:")
    imprimir_matriz(matriz)

if __name__ == "__main__":
    main()
```

Criar um novo programa.

1- Inserir elementos em uma lista
    a) Inserir
    b) Inserir em um posição
2- Pesquisar um elemento na lista
    a) Pelo valor
    b) Pela posição
3- Excluir um elemento da lista
    a) Pelo valor
    b) Pela posição
4 – Imprimir o vetor
0 – Encerrar o programa.